

LAPORAN PRAKTIKUM
STRUKTUR DATA
PEKAN KE 8

Disusun Oleh :

Muhammad Irfan 2511531008

Dosen Pengampu : Dr Wahyudi S.T M.T

Asisten Praktikum : Muhammad Galid Avero



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
TAHUN 2025

KATA PENGANTAR

Puji syukur atas kehadiran Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya sehingga penulis dapat menyelesaikan laporan praktikum dari mata kuliah Struktur Data dengan judul Shell Sort, Quick Sort dan Merge Sort dengan baik dengan benar

Laporan ini disusun sebagai bentuk hasil kegiatan praktikum Struktur Data yang di mana membahas tentang bahasa pemrograman java dan penggunaan Shell Sort, Quick Sort dan Merge Sort serta kegunaan masing-masing sorting tersebut serta cara penerapannya.

Penulis menyadari bahwa laporan praktikum ini masih jauh dari sempurna baik dari segi penulisannya dan dari segi pembahasannya. Oleh karena itu saya ingin mengucapkan terima kasih kepada dosen pengampu, asisten praktikum yang telah membimbing serta rekan-rekan yang telah membantu dalam proses penyusunan laporan ini.

Akhir kata, penulis ucapkan terima kasih pada dosen pengampu yang telah memberikan arahan dan semoga laporan ini dapat bermanfaat baik bagi penulis maupun pembaca dalam pemahaman tentang Shell Sort, Quick Sort dan Merge Sort

DAFTAR ISI

DAFTAR ISI.....	2
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	1
BAB II PEMBAHASAN	2
1.Class ShellSort_2511531008	2
Kode program	2
2. Class QuickSort_2511531008	3
Kode program	3
Output	5
3.Class MergeSort_2511531008.....	5
Kode program	5
Ouput.....	8
4. SelectionSortGUI_2511531008	9
kodeprogram	9
Output	14
BAB III PENUTUP	15
3.1 Kesimpulan	15
TUGAS PEKAN 8.....	16
1.Class Sorting_2511531008	16
OUPUT	18
ALASAN MEMILIH SHELL SORT	18

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan komponen yang paling penting untuk menyimpan nilai dan mengelola nilai secara baik dan efisien. Pengurutan nilai dan data sangatlah penting dalam dunia pemrograman. Pengurutan dalam dunia pemrograman berfungsi untuk mengurutkan nilai agar mudah di baca, analisis dan dikelola oleh user

Oleh karena itu, dibuatkan cara pengurutan dan pengelolaan data dengan baik dengan benar. Terdapat tiga algoritma untuk melakukan pengurutan pada suatu data. Yakni Shell Sort, Quick Sort dan Merge Sort. Ketiga algoritma ini menggunakan perulangan array dengan menggunakan while dan for loop. . diharapkan mahasiswa dapat memahami ketiga algoritma tersebut dan dapat diimplementasikan ke dalam aplikasi

1.2 Tujuan Praktikum

Tujuan utama dari praktikum ini adalah memberikan pemahaman kepada mahasiswa tentang Konsep Shell Sort, Quick Sort dan Merge Sort serta implementasi di dalam pemrograman bahasa Java. Selain itu bagaimana pengaplikasian berbagai berbagai-macam problem dengan menggunakan Shell Sort, Quick Sort dan Merge Sort

1.3 Manfaat Praktikum

Manfaat utama dari praktikum ini adalah mahasiswa diharapkan dapat memahami konsep, Shell Sort, Quick Sort dan Merge Sort Selain itu, mahasiswa juga diharapkan untuk dapat mengolah dan meningkatkan analisis terhadap problem dengan menggunakan ketiga algoritma tersebut..

BAB II

PEMBAHASAN

1. Class ShellSort_2511531008

Kode program

```
1 package pekan8_2511531008;
2
3 public class ShellSort_2511531008 {
4
5     public static void ShellSort(int[] A_1008) {
6         int n_1008 = A_1008.length;
7         int gap_1008 = n_1008 / 2;
8         while(gap_1008 > 0) {
9             for(int i_1008 = gap_1008; i_1008 < n_1008; i_1008++) {
10                 int temp = A_1008[i_1008];
11                 int j = i_1008;
12                 while( j >= gap_1008 && A_1008[j-gap_1008] > temp ) {
13                     A_1008[j] = A_1008[j-gap_1008];
14                     j = j-gap_1008;
15                 }
16                 A_1008[j] = temp;
17             }
18             gap_1008 = gap_1008/2;
19         }
20     }
```

Penjelasan :

- Dimethod Shellsort_2511531008 merupakan method untuk mengurutkan data / nilai
- n_1008 merupakan nilai dari panjang array, sedangkan gap_1008 adalah setengah dari panjang array
- Menggunakan perulangan for dan while
- While akan berulang selama gap_1008 lebih dari 0, gap akan selalu dibagi 2. Jadi gap akan berkurang dari 5 -> 2 -> 1 -> 0
- Perulangan for ini menyisir elemen array dimulai dari indeks yang sama dengan nilai gap sampai ke ujung akhir array.
- Temp adalah Menyimpan sementara nilai elemen saat ini
- Variabel j untuk melacak posisi indeks saat membandingkan ke belakang.

```

21 public static void main (String[] args) {
22     int[] data_1008 = {3,10,4,6,8,9,7,2,1,5};
23     System.out.print("Sebelum: ");
24     printarray(data_1008);
25     ShellSort(data_1008);
26     System.out.print("Sesudah (shell Sort): ");
27     printarray(data_1008);
28 }
29 public static void printarray(int[] arr_1008) {
30     for(int i : arr_1008) System.out.print(i + " ");
31     System.out.println();
32 }
33 }

```

Penjelasan

- Fungsi utama menjalankan program dengan mengurutkan angka acak dan tidak beraturan.
- Menggunakan method printArray untuk mencetak nilai array
- Setelah itu arr tersebut diurutkan dengan menggunakan method shellsort
- Data yang sudah terurut, kemudian ditampilkan dengan menggunakan method printArray

Ouput:

```

<terminated> ShellSort_2511531008 [Java Application] C:\Use
Sebelum: 3 10 4 6 8 9 7 2 1 5
Sesudah (shell Sort): 1 2 3 4 5 6 7 8 9 10

```

2. Class QuickSort_2511531008

Kode program

```

4 static void swap_1008(int[] arr_1008, int i_1008, int j_1008) {
5     int temp_1008 = arr_1008[i_1008];
6     arr_1008[i_1008] = arr_1008[j_1008];
7     arr_1008[j_1008] = temp_1008;
8 }
9 static void mediaOfThree_1008 (int[] arr_1008 ,int low_1008,int high_1008) {
10     int mid_1008 = low_1008 +(high_1008 - low_1008)/2;
11
12     //urutkan elemen low_1008, mid_1008, dan high_1008
13     if(arr_1008[low_1008] > arr_1008[mid_1008]) swap_1008(arr_1008,low_1008,mid_1008);
14     if(arr_1008[low_1008] > arr_1008[high_1008]) swap_1008(arr_1008,low_1008,high_1008);
15     if(arr_1008[mid_1008] > arr_1008[high_1008]) swap_1008(arr_1008,mid_1008,high_1008);
16     swap_1008(arr_1008,mid_1008,high_1008);
17 }
18

```

Penjelasan :

- program ini adalah mengurutkan data dengan menggunakan algortima QuickShort

- terdapat method swap_1008 berfungsi untuk memindahkan nilai 2 variabel. Nilai variabel A di pindahkan ke variabel B. Begitu juga dengan sebaliknya
- method mediaOfthree memeriksa nilai paling kiri, tengah dan paling kanan. Ambil nilai median diantara 3 tersebut. Setelah itu nilai median di pindahkan paling kanan sebuah array yang berfungsi sebagai pivot dengan menggunakan swap

```

19- static int partition_1008(int[] arr_1008, int low_1008, int high_1008) {
20-     //panggil fungsi mediaOfThree sebelum menentukan pi_1008vot_1008
21-     mediaOfThree_1008(arr_1008,low_1008,high_1008);
22-     int pivot_1008 = arr_1008[high_1008];
23-     int i_1008 = (low_1008-1);
24-
25-     for(int j_1008 = low_1008; j_1008 <= high_1008-1; j_1008++) {
26-         //jika elemen saat ini lebih kecil dari atau sama dengan pi_1008vot_1008
27-         if(arr_1008[j_1008] < pivot_1008) {
28-             i_1008++;
29-             swap_1008(arr_1008,i_1008,j_1008);
30-         }
31-     }
32-     swap_1008(arr_1008,i_1008+1,high_1008);
33-     return(i_1008+1);
34- }
35- static void quickSort_1008(int[] arr_1008,int low_1008, int high_1008) {
36-     if(low_1008 < high_1008) {
37-         int pi_1008 = partition_1008(arr_1008,low_1008,high_1008);
38-         quickSort_1008(arr_1008,low_1008,pi_1008-1);
39-         quickSort_1008(arr_1008,pi_1008+1,high_1008);
40-     }
41- }

```

Penjelasan :

- method partition berfungsi untuk menata ulang nilai array dengan menggunakan pivot
- mediaOfThree_1008 dipanggil untuk meletakkan angka median di indeks high_1008.
- Angka di ujung kanan dijadikan nilai acuan (pivot_1008).
- Variabel i_1008 disiapkan sebagai batas pembatas untuk elemen yang nilainya lebih kecil dari pivot.
- Fungsi quicksort ini memanggil fungsinya sendiri atau disebut dengan fungsi rekursif
- Setiap fungsi quicksort ini memanggil 2 fungsi dirinya sendiri. Yaitu fungsi subarray sebelah kiri (nilai kurang dari pivot_1008) dan fungsi subarray sebelah kanan (nilai lebih dari pivot_1008). Proses ini berulang hingga array tidak bisa dibagi lagi alias tersisa satu elemen

```

42 public static void printarr_1008(int[] arr_1008) {
43     for(int i_1008 = 0; i_1008 < arr_1008.length; i_1008++) {
44         System.out.print(arr_1008[i_1008] + " ");
45     }
46     System.out.println();
47 }
48 public static void main(String[] args) {
49     int[] arr_1008 = {10, 7, 8, 9, 1, 5};
50     int N_1008 = arr_1008.length;
51     System.out.print("Data sebelum terurut: ");
52     printarr_1008(arr_1008);
53
54     quickSort_1008(arr_1008, 0, N_1008-1);
55     System.out.print("Data terurut quickSort_1008: ");
56     printarr_1008(arr_1008);
57 }
58 }

```

Penjelasan :

- Fungsi utama menjalankan program dengan mengurutkan angka acak dan tidak beraturan.
- Diawali dengan n_1008 = panjang array, kemudian mencetak arr yang belum terurut dari berindeks 0 hingga n-1 dengan menggunakan method printarr_1008.
- Setelah itu arr tersebut diurutkan dengan menggunakan metode Quicksort
- Data yang sudah terurut, kemudian ditampilkan dengan method printarr_1008

Output

```

<terminated> QuickSort_2511531008 [Java Application] C:\U
Data sebelum terurut: 10 7 8 9 1 5
Data terurut quickSort_1008: 1 5 7 8 9 10

```

3.Class MergeSort_2511531008

Kode program

```

void sort(int arr[], int l, int r) {
    if(l < r) {
        // find the middle point
        int m = (l + r) / 2;
        // sort first and second halves
        sort(arr, l, m);

        sort(arr, m + 1, r);
        // merge the sorted halves
        merge_1008(arr, l, m, r);
    }
}

```


Penjelasan:

- Method ini adalah algoritma mergesort yang membagi elemen – elemen menjadi satu, kemudian diurutkan hingga menjadi array yang utuh. Metode ini sering disebut dengan divide and conquer
- Fungsi sort ini adalah fungsi rekursif, fungsi ini membagi panjang array menjadi 2 bagian (kanan, kiri). Proses ini terus berulang hingga fungsi ini mengembalikan 1 elemen array.

```
4 void merge_1008(int arr_1008[], int l_1008, int m_1008, int r_1008) {  
5     // find sizes of two subarrays to be merged  
6     int n1_1008 = m_1008 - l_1008 + 1;  
7     int n2_1008 = r_1008 - m_1008;  
8  
9     // Create temp arrays  
10    int L[] = new int[n1_1008];  
11    int R[] = new int[n2_1008];  
12  
13    // copy data to temp arrays  
14    for(int i_1008 = 0; i_1008 < n1_1008; ++i_1008) {  
15        L[i_1008] = arr_1008[l_1008 + i_1008];  
16    }  
17  
18    for(int j_1008 = 0; j_1008 < n2_1008; ++j_1008) {  
19        R[j_1008] = arr_1008[m_1008 + 1 + j_1008];  
20    }  
21 }
```

Penjelasan :

- Fungsi merge_1008 ini diawali dengan inisialisasi variabel. n1_1008 berfungsi untuk nilai panjang subarray sebelah kiri. Sedangkan n2_1008 berfungsi untuk nilai panjang subarray sebelah kanan.
- Array L berfungsi menyimpan nilai array sementara di kiri sedangkan array R sebelah kanan.
- Masukkan nilai array ke subarray L dan R dengan menggunakan for looping

```

25-    int k_1008 = l_1008;
26-    while(i_1008 < n1_1008 && j_1008 < n2_1008) {
27-        if (L[i_1008] <= R[j_1008]) {
28-            arr_1008[k_1008] = L[i_1008];
29-            i_1008++;
30-        } else {
31-            arr_1008[k_1008] = R[j_1008];
32-            j_1008++;
33-        }
34-        k_1008++;
35-    }
36-
37-    // copy remaining elements of L[] if any
38-    while (i_1008 < n1_1008) {
39-        arr_1008[k_1008] = L[i_1008];
40-        i_1008++;
41-        k_1008++;
42-    }
43-    // copy remaining elements of R[] if any
44-    while (j_1008 < n2_1008) {
45-        arr_1008[k_1008] = R[j_1008];
46-        j_1008++;
47-        k_1008++;
48-    }
49-}

```

Penjelasan :

- Nilai K_1008 adalah nilai dari L_1008
- Elemen pada array L_1008 dan R_1008 dibandingkan satu per satu dari depan. angka yang lebih kecil, angka itulah yang dimasukkan kembali ke array asli (arr_1008[k_1008]).
- Copy sisa elements array L_1008 dan array R_1008 ke dalam array_1008 jika salah satu array sudah habis duluan.

```

64-    static void printArray_1008(int arr[]) {
65-        int n = arr.length;
66-        for(int i = 0; i < n; ++i) {
67-            System.out.print(arr[i] + " ");
68-        }
69-        System.out.println();
70-    }
71-
72-    public static void main(String[] args) {
73-        int arr[] = {12, 11, 13, 5, 6, 7};
74-        System.out.println("Sebelum terurut");
75-        printArray_1008(arr);
76-
77-        MergeSort_2511531008 ob = new MergeSort_2511531008();
78-        ob.sort(arr, 0, arr.length - 1);
79-
80-        System.out.println("\nSesudah Terurut menggunakan merge sort");
81-        printArray_1008(arr);
82-    }
83-}

```

Penjelasan :

- Fungsi utama menjalankan program dengan mengurutkan angka acak dan tidak beraturan.
- Diawali mencetak arr yang belum terurut dari berindeks 0 hingga n-1.
- Setelah itu arr tersebut diurutkan dengan menggunakan method Merge sort
- Data yang sudah terurut, kemudian ditampilkan dengan menggunakan method printArray_1008

Ouput

```
<terminated> MergeSort_2511531008 [Java Applicati  
Sebelum terurut  
12 11 13 5 6 7  
  
Sesudah Terurut menggunakan merge sort  
5 6 7 11 12 13
```

4. SelectionSortGUI_2511531008

kodeprogram

```
1 package pekan7_2511531008;
2
3 import java.awt.BorderLayout;
4 import java.awt.Color;
5 import java.awt.Dimension;
6 import java.awt.FlowLayout;
7 import java.awt.Font;
8
9 import javax.swing.JFrame;
10 import javax.swing.JPanel;
11 import javax.swing.JLabel;
12 import javax.swing.JOptionPane;
13 import javax.swing.BorderFactory;
14 import javax.swing.JButton;
15 import javax.swing.JTextField;
16 import javax.swing.SwingConstants;
17 import javax.swing.SwingUtilities;
18 import javax.swing.JTextArea;
19 import javax.swing.JScrollPane;
20
21 public class InsertionSortGUI_2511531008 extends JFrame {
22     private static final long serialVersionUID = 1L;
23     private int[] array_1008;
24     private JLabel[] labelArray_1008;
25     private JButton stepButton_1008, resetButton_1008, setButton_1008;
26     private JTextField inputField_1008;
27     private JPanel panelArray_1008;
28     private JTextArea stepArea_1008;
29
30     private int i_1008 = 1, j_1008;
31     private boolean sorting_1008 = false;
32     private int stepCount_1008 = 1;
33 }
```

Penjelasan :

- Dari baris 3- 19 adalah library dari java Swing yang menyediakan GUI textfield, tombol, layout dan lain lain
- Baris 22- 32 adalah mendefenisikan nama variabel dan nama – nama dari tombol, textfield, panel dan lain lain

```
34 public BubbleSortGUI_2511531008() {
35     setTitle("Bubble Sort Langkah per Langkah");
36     setSize(750, 400);
37     setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
38     setLocationRelativeTo(null);
39     setLayout(new BorderLayout());
40
41     // Panel input
42     JPanel inputPanel_1008 = new JPanel(new FlowLayout());
43     inputField_1008 = new JTextField(30);
44     setButton_1008 = new JButton("Set Array");
45     inputPanel_1008.add(new JLabel("Masukkan angka (pisahkan dengan koma :)"));
46     inputPanel_1008.add(inputField_1008);
47     inputPanel_1008.add(setButton_1008);
48 }
```

Penjelasan

- Baris 35-39 adalah kode untuk menampilkan tampilan GUI dengan title “Bubble sort langkah per langkah”
- Baris 42-47 merupakan kode untuk menampilkan GUI panel input, yang terdiri dengan inputfield, setbutton

```
49      // Panel array visual
50      panelArray_1008 = new JPanel();
51      panelArray_1008.setLayout(new FlowLayout());
52
53      // Panel kontrol
54      JPanel controlPanel_1008 = new JPanel();
55      stepButton_1008 = new JButton("Langkah Selanjutnya");
56      resetButton_1008 = new JButton("Reset");
57      stepButton_1008.setEnabled(false);
58      controlPanel_1008.add(stepButton_1008);
59      controlPanel_1008.add(resetButton_1008);
60
61      // Area teks untuk log langkah
62      stepArea_1008 = new JTextArea(0, 60);
63      stepArea_1008.setEditable(false);
64      stepArea_1008.setFont(new Font("Monospaced", Font.PLAIN, 14));
65      JScrollPane scrollPane_1008 = new JScrollPane(stepArea_1008);
66  }
```

Penjelasan :

- Baris 49- 65 adalah kode untuk menampilkan tampilan GUI terdiri dengan panel array, kontrol dan teks langkah - langkah
- Jenis font yang digunakan untuk GUI adalah monospaces dengan ukuran 14

```
67      // Tambahkan panel ke frame
68      add(inputPanel_1008, BorderLayout.NORTH);
69      add(panelArray_1008, BorderLayout.CENTER);
70      add(controlPanel_1008, BorderLayout.SOUTH);
71      add(scrollPane_1008, BorderLayout.EAST);
72
73      // Event Set Array
74      setButton_1008.addActionListener(e -> setArrayFromInput_1008());
75
76      // Event Langkah Selanjutnya
77      stepButton_1008.addActionListener(e -> performStep_1008());
78
79      // Event reset_1008
80      resetButton_1008.addActionListener(e -> reset_1008());
81  }
```

Penjelasan :

- Baris 67-71 adalah fitur GUI untuk menambahkan tampilan inputpanel, controlpanel serta scrollpane.

- Baris 73 – 80 adalah fitur untuk mengarahkan ke 3 event yakni setArray input, perfromStep dan reset

```

83 private void setArrayFromInput() {
84     String text_1008 = inputField_1008.getText().trim();
85     if (text_1008.isEmpty()) return;
86     String[] parts = text_1008.split(",");
87     array_1008 = new int[parts.length];
88     try {
89         for (int k_1008 = 0; k_1008 < parts.length; k_1008++) {
90             array_1008[k_1008] = Integer.parseInt(parts[k_1008].trim());
91         }
92     } catch (NumberFormatException e) {
93         JOptionPane.showMessageDialog(this, "Masukkan hanya angka yang dipisahkan"
94             + " dengan koma!", "ERROR", JOptionPane.ERROR_MESSAGE);
95     }
96     i_1008 = 1;
97     stepCount_1008 = 1;
98     sorting_1008 = true;
99     stepButton_1008.setEnabled(true);
100    stepArea_1008.setText("");
101    panelArray_1008.removeAll();
102    labelArray_1008 = new JLabel[array_1008.length];
103    for (int k_1008 = 0; k_1008 < array_1008.length; k_1008++) {
104        labelArray_1008[k_1008] = new JLabel(String.valueOf(array_1008[k_1008]));
105        labelArray_1008[k_1008].setFont(new Font("Arial", Font.BOLD, 24));
106        labelArray_1008[k_1008].setBorder(BorderFactory.createLineBorder(Color.BLACK));
107        labelArray_1008[k_1008].setPreferredSize(new Dimension(50, 50)); //  Diperbaiki
108        labelArray_1008[k_1008].setHorizontalAlignment(SwingConstants.CENTER);
109        panelArray_1008.add(labelArray_1008[k_1008]);
110    }
111    panelArray_1008.revalidate();
112    panelArray_1008.repaint();
113 }

```

Penjelasan :

- Program mengambil teks tersebut, lalu memotongnya berdasarkan tanda koma lewat fungsi `.split(",")`.
- Program mengubah potongan teks menjadi tipe data angka (`Integer.parseInt`) dan memasukkannya ke dalam array `array_1008`. Jika input bukan angka, kotak pesan *Error* (`JOptionPane`) akan muncul.
- Kotak visual (`JLabel`) dibuat sebanyak jumlah angka yang dimasukkan dan langsung ditampilkan di tengah layar dengan garis pembatas hitam.
- Tombol "Langkah Selanjutnya" diaktifkan (`setEnabled(true)`).

```

119 private void performStep_1008() {
120     if (!sorting_1008 || i_1008 >= array_1008.length - 1) {
121         sorting_1008 = false;
122         stepButton_1008.setEnabled(false);
123         JOptionPane.showMessageDialog(this, "Sorting selesai!");
124         return;
125     }
126
127     StringBuilder stepLog_1008 = new StringBuilder();
128     labelArray_1008[j_1008].setBackgroundColor(Color.CYAN);
129     labelArray_1008[j_1008 + 1].setBackgroundColor(Color.CYAN);
130     if (array_1008[j_1008] > array_1008[j_1008 + 1]) {
131         // Swap
132         int temp_1008 = array_1008[j_1008];
133         array_1008[j_1008] = array_1008[j_1008 + 1];
134         array_1008[j_1008 + 1] = temp_1008;
135         labelArray_1008[j_1008].setBackgroundColor(Color.RED);
136         labelArray_1008[j_1008 + 1].setBackgroundColor(Color.RED);
137         stepLog_1008.append("Langkah ").append(stepCount_1008).append(": Menukar elemen ke-")
138             .append(j_1008).append(" ").append(array_1008[j_1008 + 1]).append(" dengan elemen ke-")
139             .append(j_1008 + 1).append(" ").append(array_1008[j_1008]).append("\n");
140     } else {
141         stepLog_1008.append("Langkah ").append(stepCount_1008).append(": Tidak ada pertukaran antara ke-")
142             .append(j_1008).append(" dan ke-").append(j_1008 + 1).append("\n");
143     }

```

Penjelasan:

- Sebelum dimulai program akan mengecek apakah programnya sorting_1008 selesai atau i_1008 melebihi panjang array. Jika iya maka program selesai
- Baris 127 – 129 merupakan mengganti latar belakang menjadi warna cyan saat kedua elemen aktif dibandingkan
- Baris 130 -139 merupakan pergantian atau swap antara 2 elemen berdekatan. Kemudian ditampilkan langkah – langkah pertukaran nilai di dalam GUI
- Baris 140-143 jika indeks yang pilih tidak ada 2 elemen berdekatan. Maka lakukan langkah ini

```

144     stepLog_1008.append("Hasil: ").append(arrayToString_1008(array_1008)).append("\n\n");
145     stepArea_1008.append(stepLog_1008.toString());
146     updateLabels_1008();
147     j_1008++;
148     if (j_1008 >= array_1008.length - i_1008 - 1) {
149         j_1008 = 0;
150         i_1008++;
151     }
152     stepCount_1008++;
153     if (i_1008 >= array_1008.length - 1) {
154         sorting_1008 = false;
155         stepButton_1008.setEnabled(false);
156         JOptionPane.showMessageDialog(this, "Sorting selesai!");
157     }
158 }

```

Penjelasan:

- Teks log yang sudah dibuat dimasukkan ke dalam stepArea_1008 agar riwayat tercetak di layar.
- Fungsi updateLabels_1008() dipanggil untuk memperbarui teks angka baru

- `j_1008++` Menggeser pointer satu langkah ke kanan untuk perbandingan berikutnya pada klik selanjutnya.

```

160 private void updateLabels_1008() {
161     for (int k_1008 = 0; k_1008 < array_1008.length; k_1008++) {
162         labelArray_1008[k_1008].setText(String.valueOf(array_1008[k_1008]));
163     }
164 }
165
166 private void reset_1008() {
167     inputField_1008.setText("");
168     panelArray_1008.removeAll();
169     panelArray_1008.revalidate();
170     panelArray_1008.repaint();
171     stepArea_1008.setText("");
172     stepButton_1008.setEnabled(false);
173     sorting_1008 = false;
174     i_1008 = 0;
175     j_1008 = 0;
176     stepCount_1008 = 1;
177 }
178
179 private String arrayToString_1008(int[] arr_1008) {
180     StringBuilder sb = new StringBuilder();
181     for (int k_1008 = 0; k_1008 < arr_1008.length; k_1008++) {
182         sb.append(arr_1008[k_1008]);
183         if (k_1008 < arr_1008.length - 1) sb.append(",");
184     }
185     return sb.toString();
186 }
187
188 public static void main(String[] args) {
189     SwingUtilities.invokeLater(() -> {
190         BubbleSortGUI_2511531008 gui = new BubbleSortGUI_2511531008();
191         gui.setVisible(true);
192     });
193 }
194 }

```

Penjelasan Singkat:

- Method `updateLabels` berfungsi untuk mengupdate labels dari tampilan GUI menjadi memori di dalam program dengan menggunakan `setText`
- Method `reset` adalah method untuk menghapus semua elemen, termasuk langkah-langkah, dan semua nilai yang terdapat pada tampilan GUI
- Method `String arrayToString` merupakan emthod untuk mengubah susunan array menjadi sebuah string dengan konversi tipe data. Konversi tipe data yang digunakan disini ialah `sb.toString()`;

Output

Masukkan angka (pisahkan dengan koma) :

1	2	3	4
	5	7	

Langkah 1: Tidak ada pertukaran antara ke-0 dan ke-1
Hasil: 3,4,7,2,1,5

Langkah 2: Tidak ada pertukaran antara ke-1 dan ke-2
Hasil: 3,4,7,2,1,5

Langkah 3: Menukar elemen ke-2 (7) dengan elemen ke-3 (2)
Hasil: 3,4,2,7,1,5

Langkah 4: Menukar elemen ke-3 (7) dengan elemen ke-4 (1)
Hasil: 3,4,2,1,7,5

Langkah 5: Menukar elemen ke-4 (7) dengan elemen ke-5 (5)
Hasil: 3,4,2,1,5,7

BAB III

PENUTUP

3.1 Kesimpulan

Dari praktikum ini dapat disimpulkan bahwa sorting merupakan struktur data linear yang menyimpan elemen secara ascending dan descending. Dalam praktikum ini kita berhasil menggunakan 3 macam algoritma sorting yakni Quick sort, shell sort, dan merge sort. Dari ketiga algoritma ini, algoritma yang paling bagus dipakai ialah algoritma Quick sort karena kompleksitas waktunya jauh lebih kecil dibandingkan 2 algoritma tersebut. Diharapkan Mahasiswa dapat mengimplementasikan algoritma tersebut dalam kehidupan sehari-hari.

TUGAS PEKAN 8

1.Class Sorting_2511531008

```
1 package pekan8_2511531008;
2 import java.util.Scanner;
3
4 class Lagu_1008 {
5     String judul;
6     String penyanyi;
7     int durasi;
8
9     public Lagu_1008(String judul, String penyanyi, int durasi) {
10         this.judul = judul;
11         this.penyanyi = penyanyi;
12         this.durasi = durasi;
13     }
14     @Override
15     public String toString() {
16         return String.format("%-25s | %-16s | %d detik", judul, penyanyi, durasi);
17     }
18 }
19
```

Penjelasan :

- Membuat konstruktor Lagu_1008
- Konstruktor ini terdiri dari beberapa attribute seperti nama, nim dan prodi
- Override berfungsi untuk menampilkan konstruktor sesuai format tertentu

```
--
20 public class Sorting_2511531008 {
21     //array maksimal 20 lagu
22     static Lagu_1008[] dataLagu_1008 = new Lagu_1008[20];
23     static int jumlahLagu_1008 = 0;
24
25     //method isi lagu (min 7 buah)
26     static void inputData_1008() {
27         dataLagu_1008[0] = new Lagu_1008("Faded", "Alan walker ", 210);
28         dataLagu_1008[1] = new Lagu_1008("Alone", "Alan walker", 195);
29         dataLagu_1008[2] = new Lagu_1008("Ghost", "Justin bieber", 183);
30         dataLagu_1008[3] = new Lagu_1008("Terlalu lama", "vierra", 220);
31         dataLagu_1008[4] = new Lagu_1008("alone", "marshmello", 240);
32         dataLagu_1008[5] = new Lagu_1008("celengan rindu", "lupa", 198);
33         dataLagu_1008[6] = new Lagu_1008("Rang Talu", "Anas Mardin", 175);
34         jumlahLagu_1008 = 7;
35     }
36
```

Penjelasan :

- Baris 22-23 inisiasi objek Lagu_1008 dan variabel jumlahLagu_1008 = 0;
- Baris 26-35 mengisi 7 buah lagu dengan masing masing atribut berupa judul, penyanyi dan durasi

```

static void shellSort_1008() {
    int gap = jumlahLagu_1008 / 2;
    while (gap > 0) {
        for (int i = gap; i < jumlahLagu_1008; i++) {
            Lagu_1008 temp = dataLagu_1008[i];
            int j = i;
            while (j >= gap && dataLagu_1008[j - gap].judul_1008.compareToIgnoreCase(temp.judul_1008) > 0) {
                dataLagu_1008[j] = dataLagu_1008[j - gap];
                j -= gap;
            }
            dataLagu_1008[j] = temp;
        }
        gap /= 2;
    }
}
...

```

- Menggunakan perulangan for dan while
- While akan berulang selama gap_1008 lebih dari 0, gap akan selalu dibagi 2. Jadi gap akan berkurang dari 5 -> 2 -> 1 -> 0
- Perulangan for ini menyisir elemen array dimulai dari indeks yang sama dengan nilai gap sampai ke ujung akhir array.
- Temp adalah Menyimpan sementara nilai elemen saat ini
- Variabel j untuk melacak posisi indeks saat membandingkan ke belakang.

```

53 public static void main(String[] args) {
54     System.out.println("\n=== Sorting Playlist NIM: 2511531008 ===");
55     System.out.println("Algoritma Dipilih: Shell Sort (judul_1008 A-Z)");
56
57     inputData_1008();
58
59     System.out.println("\nData Sebelum Sorting:");
60     System.out.println("-----");
61     for (int i = 0; i < jumlahLagu_1008; i++) {
62         System.out.println((i + 1) + ". " + dataLagu_1008[i]);
63     }
64
65     // Jalankan Shell Sort
66     shellSort_1008();
67
68     // Tampilkan sesudah sorting
69     System.out.println("\nData Setelah Shell Sort (judul_1008 A-Z):");
70     System.out.println("-----");
71     for (int i = 0; i < jumlahLagu_1008; i++) {
72         System.out.println((i + 1) + ". " + dataLagu_1008[i]);
73     }
74
75     System.out.println("\nSorting selesai!");
76 }
77 }

```

- Fungsi utama ini membuat playlist lagu yang belum terurut
- Kemudian di urutan dengan method shellSort_1008. Kemudian baru ditampilkan playlist lagu yang sudah diurutkan dengan algoritma shellsort

OUPUT

```
Console X
<terminated> Sorting_2511531008 [Java Application] C:\Users\rifqi\.p2\pool\plugins\org.e

=== Sorting Playlist NIM: 2511531008 ===
Algoritma Dipilih: Shell Sort (judul_1008 A-Z)

Data Sebelum Sorting:
-----
1. Faded           | Alan walker   | 210 detik
2. Alone           | Alan walker   | 195 detik
3. Ghost           | Justin bieber | 183 detik
4. Terlalu lama    | vierra        | 220 detik
5. alone           | marshmello    | 240 detik
6. celengan rindu  | lupa          | 198 detik
7. Rang Talu       | Anas Mardin   | 175 detik

Data Setelah Shell Sort (judul_1008 A-Z):
-----
1. Alone           | Alan walker   | 195 detik
2. alone           | marshmello    | 240 detik
3. celengan rindu  | lupa          | 198 detik
4. Faded           | Alan walker   | 210 detik
5. Ghost           | Justin bieber | 183 detik
6. Rang Talu       | Anas Mardin   | 175 detik
7. Terlalu lama    | vierra        | 220 detik

Sorting selesai!
```

ALASAN MEMILIH SHELL SORT

Alasan saya memilih Shell Sort karena algoritma ini lebih mudah dipahami dibandingkan quicksort dan mergesort. Selain itu algoritma shell sort lebih pendek syntax dan tidak menggunakan fungsi rekursif di bandingan dengan quicksort dan mergesort.

Kompleksitas Shell sort

Worst case : n^2 terjadi ketika data tidak ada terurut sama sekali dan acak

Best Case : $n \log n$ terjadi ketika semua data sudah terurut atau data hampir terurut